

Estrutura de Dados

Vetores e Listas

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Vetor: Bloco Contíguo

O vetor é uma sequência de elementos do mesmo tipo armazenados em posições vizinhas da memória, sem intervalos. Essa contiguidade não é detalhe de implementação: é a propriedade que permite calcular a localização de qualquer elemento em vez de procurá-la, e dela decorrem todas as vantagens e limitações da estrutura.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Acesso por Índice em Tempo Constante

Conhecendo o endereço inicial, o índice desejado e o espaço ocupado por elemento, o endereço do elemento procurado é obtido por uma única multiplicação seguida de uma soma. Nenhum percurso é necessário, e por isso alcançar a primeira posição custa exatamente o mesmo que alcançar a milésima.



Prof. Everaldo Graciliano Souza Ribeiro

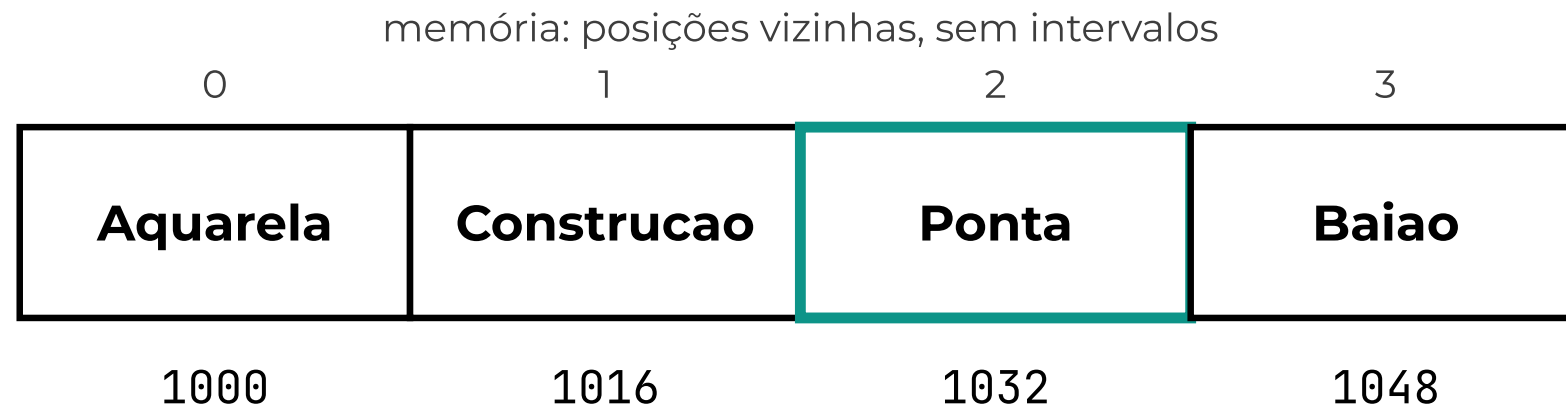
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Vetor Calcula, Não Procura



$$\text{endereço} = 1000 + 2 \times 16 = 1032$$

base + índice × tamanho — uma conta, nenhum percurso

Chegar ao índice 2 custa **exatamente o mesmo** que chegar ao índice 2 000.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa



everaldoribeiro01@gmail.com



<https://www.linkedin.com/in/everaldoribeiroads>

A Lista do Python é um Vetor que Cresce

A lista do Python não tem tamanho fixo, mas por dentro continua sendo um bloco contíguo. A linguagem reserva um bloco com folga e, quando ele se esgota, aloca outro maior e transfere os elementos. O crescimento aparentemente ilimitado é, na verdade, uma sequência de blocos sucessivamente substituídos.



Prof. Everaldo Graciliano Souza Ribeiro

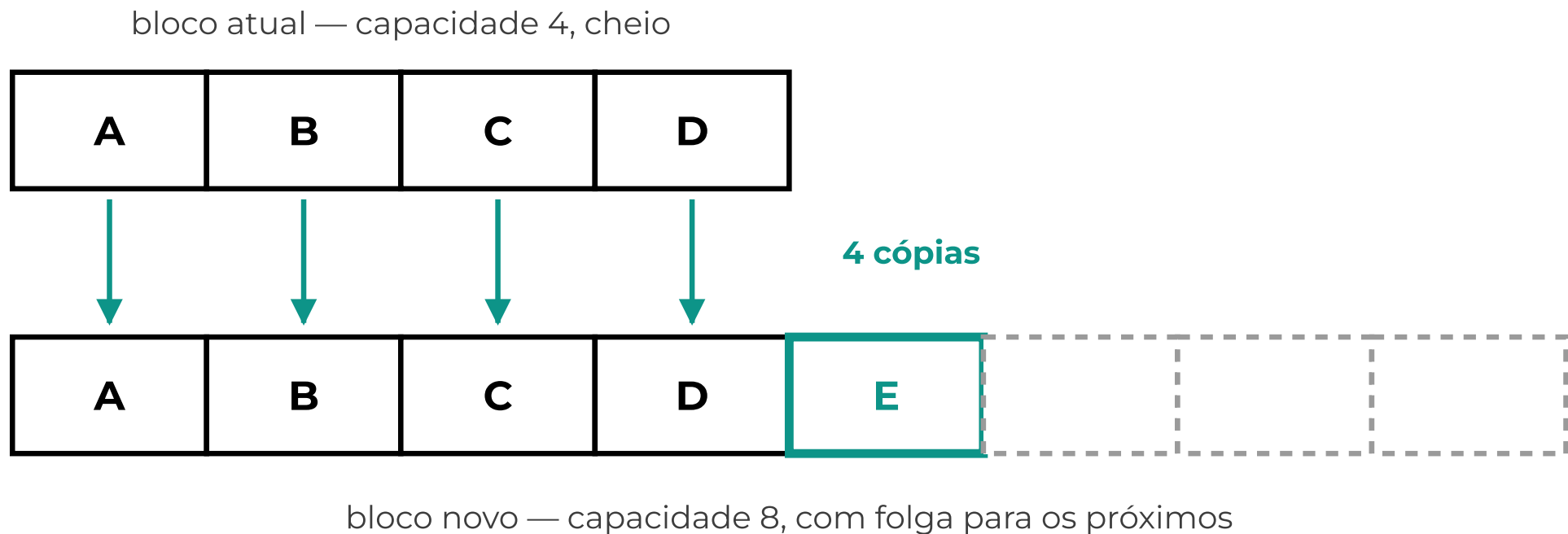
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Bloco Enche e Precisa Mudar



Quando o bloco enche, aloca-se outro maior e **copia-se tudo**. Por isso inserir no fim é barato *em média*, não sempre.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserir no Fim × Inserir no Meio

Acrescentar ao fim aproveita o espaço já reservado e não perturba nenhum elemento existente. Inserir numa posição intermediária, ao contrário, obriga a empurrar todos os elementos seguintes uma casa adiante para abrir espaço. A diferença entre as duas operações cresce proporcionalmente ao tamanho da estrutura.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

A Playlist e a Importância da Ordem

O acervo apenas registra o que existe, sem compromisso com sequência. A playlist é diferente: nela a posição de cada faixa é informação relevante, porque determina a ordem de reprodução. É a primeira estrutura do projeto em que inserir na terceira posição significa algo distinto de inserir na última.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Mesmo Contrato, Outra Implementação

A Playlist declara operações de acrescentar, inserir em posição, remover de posição e obter por índice. Hoje essas operações se apoiam na lista do Python. Na próxima aula a mesma interface será reconstruída com nós encadeados, e nenhum trecho do programa principal precisará ser alterado por causa disso.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Quando o Vetor é a Escolha Certa

O vetor compensa quando o acesso por índice é frequente, o número de elementos varia pouco e as inserções ocorrem predominantemente no fim. Perde quando há muitas inserções e remoções em posições intermediárias. Reconhecer a qual desses cenários o problema pertence é o critério de escolha, não a preferência pessoal.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Playlist: Estado e Acrescentar

```
class Playlist:
    """Sequencia ordenada de faixas para reproducao."""

    def __init__(self, nome):
        self.nome = nome
        # Vetor redimensionavel. Na Aula 04 vira uma cadeia de nos.
        self._faixas = []

    def adicionar(self, faixa):
        """Acrescenta no fim. Nada e deslocado: custo constante."""
        self._faixas.append(faixa)
```



Inserir e Remover por Posição

```
def inserir_em(self, posicao, faixa):
    if not 0 ≤ posicao ≤ len(self._faixas):
        raise IndexError("posicao fora da playlist")
    # Empurra uma casa a frente TODAS as faixas dessa posicao em diante.
    self._faixas.insert(posicao, faixa)

def remover_em(self, posicao):
    if not 0 ≤ posicao < len(self._faixas):
        raise IndexError("posicao fora da playlist")
    # Fecha o buraco puxando as seguintes uma casa para tras.
    return self._faixas.pop(posicao)
```



Obter: a Vantagem do Vetor

```
def obter(self, posicao):  
    """Acesso direto por indice: custo constante, sem percorrer nada.  
  
    E a vantagem do vetor - e exatamente o que a lista encadeada  
    da Aula 04 vai perder em troca de insercoes baratas.  
    """  
    if not 0 ≤ posicao < len(self._faixas):  
        raise IndexError("posicao fora da playlist")  
    return self._faixas[posicao]
```



A Playlist no Menu

```
def inserir(acervo, playlist):  
    faixa = acervo.buscar(input("  titulo: ").strip())  
    if faixa is None:  
        print("  nao esta no acervo")  
        return  
  
    posicao = int(input(f"  posicao (0 a {len(playlist)}): "))  
    playlist.inserir_em(posicao, faixa)  
    # 0 proprio programa mostra ao aluno o custo da operacao:  
    print(f"  {len(playlist) - posicao - 1} faixas deslocadas")
```



Atividades

Exercício 1

Numa playlist de 100 faixas, inserir na posição 0 desloca quantas? E na posição 99?

Exercício 2

Por que obter a faixa do índice 500 não custa mais caro que obter a do índice 0?

Exercício 3

O que acontece por dentro quando um append encontra o bloco cheio?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Criar mover(origem, destino) na Playlist, reaproveitando as operações existentes.
2. Contar e exibir quantos deslocamentos cada inserção provocou durante a sessão.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>